M.Kumarasamy College of Engineering
NAAC Accredited Autonomous Institution
Approved by AICTE & Affiliated to Anna University
ISO 9001:2015 Certified Institution
Thalavapalayam, Karur - 639 113, TAMILNADU.

ITC1365 – EXPLORATORY DATA ANALYTICS

ADAPTIVE TELEMETRY-DRIVEN FEED-FORWARD MICRO-VENTILATION FOR WORKSTATION-SCALE INDOOR AIR MONITORING

A PROJECT REPORT

Submitted by

Mohamed Tariq M	**927624BIT066**
Sairam M S	**927624BIT091**
Srinath Rao R	**927624BIT105**
Tharun K	**927624BIT115**

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY
In
INFORMATION TECHNOLOGY

M.KUMARASAMY COLLEGE OF ENGINEERING,
KARUR – 639 113

DECEMBER 2026

M. KUMARASAMY COLLEGE OF ENGINEERING, KARUR
(Autonomous Institution affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

This is to certify that this project report on "Adaptive Telemetry-Driven Feed-Forward Micro-Ventilation for Workstation-Scale Indoor Air Monitoring" is the bonafide work of Mohamed Tariq M (927624BIT066), Sairam M S (927624BIT091), Srinath Rao R (927624BIT105), Tharun K (927624BIT115) who carried out the project work during the academic year 2026 - 2027 under my supervision.

Signature

Dr. K. RAVIKUMAR M.E., Ph.D.,
HEAD OF THE DEPARTMENT,
Department of Information Technology
M.Kumarasamy College of Engineering,
Thalavapalayam, Karur -639 113.

Signature

Mrs. S. SUBHA M.E.,
SUPERVISOR,
Assistant Professor,
Department of Information Technology
M.Kumarasamy College of Engineering,
Thalavapalayam, Karur -639 113.

This Project Work (ITC1365) report has been submitted for the 5th Semester Project viva voce Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

We jointly declare that the project report on "ADAPTIVE TELEMETRY-DRIVEN FEED-FORWARD MICRO-VENTILATION FOR WORKSTATION-SCALE INDOOR AIR MONITORING" is the result of original work done by us to the best of our knowledge, similar work has not been submitted to "ANNA UNIVERSITY CHENNAI" for the requirement of Degree of Information Technology. This project report is submitted on the partial fulfilment of the requirement of the award of Degree of Information Technology.

Signature

Mohamed Tariq M 927624BIT066 _____

Sairam M S 927624BIT091 _____

Srinath Rao R 927624BIT105 _____

Tharun K 927624BIT115 _____

PLACE: KARUR

ACKNOWLEDGEMENT

We gratefully remember our beloved Founder Chairman, (Late) Thiru. M. Kumarasamy, whose vision and legacy laid the foundation for our education and inspired us to successfully complete this project.

We extend our sincere thanks to Dr. K. Ramakrishnan, Chairman, and Mr. K. R. Charun Kumar, Joint Secretary, for providing excellent infrastructure and continuous support throughout our academic journey.

We are privileged to extend our heartfelt thanks to our respected Principal, Dr. B. S. Murugan, M.Tech., Ph.D., for providing us with a conducive environment and constant encouragement to pursue this project work.

We sincerely thank Dr.K.Ravikumar, M.E., Ph.D., Associate Professor and Head, Department of Information Technology, for his continuous support, valuable guidance and motivation throughout the course of this project.

Our special thanks and deep sense of appreciation go to our Course Coordinator, Dr.R.Dhivya, M.Tech., Ph.D., Assistant Professor, Department of Information Technology, for her exceptional guidance, continuous supervision, constructive suggestions, and unwavering support, all of which have been instrumental in the successful execution of this project.

We would also like to acknowledge Mrs. S. Subha, M.E., supervisor for her constant encouragement, technical insights, and coordination that contributed to the smooth progress and completion of our project work.

We gratefully thank all the faculty members of the Department of Information Technology for their timely assistance, valuable insights and constant support during various phases of the project.

Finally, we extend our profound gratitude to our parents and friends for their encouragement, moral support, and motivation, without which the successful completion of this project would not have been possible.

DEPARTMENT OF INFORMATION TECHNOLOGY

VISION OF THE INSTITUTION

To emerge as a leader among the top institutions in the field of technical education.

MISSION OF THE INSTITUTION

1. Produce smart technocrats with empirical knowledge who can surmount the global challenges

2. Create a diverse, fully-engaged, learner-centric campus environment to provide quality education to the students

3. Maintain mutually beneficial partnerships with our alumni, industry, and Professional associations

VISION OF THE DEPARTMENT

To Create Technically Skilled IT Professionals to meet Corporate Expectations and to Address Societal Concerns.

MISSION OF THE DEPARTMENT

1. To build competency in advanced computing technologies and implement project-based learning

2. To establish industry collaboration for research projects and consultancy.

3. To inculcate Entrepreneurial skills to serve the societal benefits.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

- **PEO1:** Graduates will be able to solve real world problems using advanced computing technologies
- **PEO2:** Encompass the ability to build innovative software products and become a successful entrepreneur

- **PEO3:** Carry out industrial projects and succeed in research careers

PROGRAM OUTCOMES (POs)

Engineering students will be able to:

1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, give and receive clear instructions.

11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

- **PSO1:** Apply knowledge of computer programming concepts to develop software projects.
- **PSO2:** Design software in a futuristic approach to adapt cutting-edge technologies.
- **PSO3:** Solve societal problems using computational intelligence.

Abstract (Key words)	**POs & PSOs Mapping**
Workstation Micro-Ventilation, IoT Telemetry, Cross-Domain Inference, Ridge Regression, Zero-Downtime Hot Reload, Autonomous Retraining, Embedded Actuation.	PO1, PO2, PO3, PO4, PO5, PO6, PO7, PO8, PO9, PO10, PO11, PSO1, PSO2, PSO3

SDG Goal	Remarks
SDG 3: Good Health and Well-Being	Mitigates localized cognitive fatigue, drowsiness, and respiratory vulnerabilities by pre-emptively preventing indoor pollutant and gas accumulation before safety thresholds are exceeded, supporting healthier personal workspace environments.
SDG 9: Industry, Innovation, and Infrastructure	Pioneers privacy-first cross-domain cyber-physical telemetry by coupling host operating system process indicators with edge IoT microcontroller actuation, running completely offline with pure-Python closed-form machine learning.
SDG 11: Sustainable Cities and Communities	Replaces high-surge, disruptive commercial HVAC cycling with continuous, whisper-quiet micro-ventilation, significantly reducing localized kinetic energy waste and lowering built-environment electrical loads.

ABSTRACT

Maintaining indoor air quality (IAQ) is crucial for human health, cognitive performance, and energy sustainability in modern sealed building environments. Conventional Demand-Controlled Ventilation (DCV) and HVAC management systems operate on a purely reactive paradigm, actuating mechanical ventilation only after toxic contaminants —such as volatile gases, carbon dioxide, or particulates—cross pre-configured regulatory safety thresholds. Due to the inherent transport delays, chamber dilution lag, and physical gas diffusion times (frequently taking 3 to 7 minutes before stationary physical sensors detect a problem), reactive control inevitably suffers from pronounced concentration overshoot, severe acoustic disturbances caused by sudden high-speed fan cycling, and significant kinetic energy waste.

To address these fundamental limitations, this project presents “Adaptive Telemetry-Driven Feed-Forward Micro-Ventilation for Workstation-Scale Indoor Air Monitoring” (AeroOS). The proposed framework couples continuous edge-level IoT telemetry with a cross-domain predictive inference model to anticipate indoor air degradation before it physically manifests. The system integrates an ESP32 microcontroller with an MQ-135 air quality gas sensor, DHT22 temperature and humidity sensing, and a PIR motion sensor, alongside host workstation OS telemetry (CPU utilization, memory usage, and compute-intensive process execution flags).

Telemetry data are streamed to a Python-based backend over local UDP sockets. The analytical core features a pure-Python closed-form Ridge Regression engine that forecasts sensor response 30 seconds ahead. An autonomous background retraining daemon periodically recalibrates model weights directly from streaming data, hot-reloading updated parameters with zero server downtime. The system pre-emptively dispatches feed-forward actuation commands to an ESP32 exhaust fan while enforcing local hardware safety threshold overrides. Experimental evaluation demonstrates that the proposed feed-forward micro-ventilation completely eliminates threshold overshoot, ensures whisper-quiet acoustic compliance (<28 dBA), and achieves a 35.7% reduction in fan kinetic energy expenditure compared with conventional reactive cycling.

TABLE OF CONTENTS

Chapter No.	TITLE	Page No.
	VISION AND MISSION OF THE INSTITUTE AND DEPARTMENT	**v**
	ABSTRACT	**1**
	TABLE OF CONTENTS	**2**
	LIST OF ABBREVIATIONS	**3**
	LIST OF FIGURES	**4**
1	**INTRODUCTION**	**5**
	1.1 INTRODUCTION	6
	1.2 NEED FOR FEED-FORWARD MICRO-VENTILATION	7
2	**LITERATURE REVIEW**	**8**
3	**PROPOSED SYSTEM**	**13**
	3.1 OBJECTIVE	14
	3.2 PROPOSED ARCHITECTURE	15
	3.3 PROPOSED METHODOLOGY	16
	3.4 MODULE DESCRIPTION	25
	3.4.1 SENSOR DATA ACQUISITION MODULE	25
	3.4.2 DATA TRANSMISSION AND BACKEND PROCESSING MODULE	25
	3.4.3 SENSOR HEALTH MONITORING MODULE	26
	3.4.4 ADAPTIVE DECISION FRAMEWORK MODULE	26
	3.4.5 PREDICTIVE STABILITY ASSESSMENT & RECOMMENDATION MODULE	27
	3.4.6 SMART AEROOS DASHBOARD AND REPORTING MODULE	27
4	**RESULT AND DISCUSSION**	**29**
5	**CONCLUSION AND FUTURE ENHANCEMENT**	**32**
	REFERENCES	**35**
	APPENDIX I	**37**
	APPENDIX II	**40**
	APPENDIX III	**46**

LIST OF ABBREVIATION

Abbreviation	Full Form
AI	Artificial Intelligence
IoT	Internet of Things
ESP32	Espressif Systems 32-bit Microcontroller
IAQ	Indoor Air Quality
DCV	Demand-Controlled Ventilation
ADC	Analog-to-Digital Converter
UDP	User Datagram Protocol
API	Application Programming Interface
PIR	Passive Infrared Sensor
CPU	Central Processing Unit
RAM	Random Access Memory
CSV	Comma-Separated Values
DFD	Data Flow Diagram
MAE	Mean Absolute Error
RMSE	Root Mean Squared Error
OS	Operating System

LIST OF FIGURES

Figure No.	Title	Page No.
3.2	Proposed Architecture	15
3.3.2	Data Flow Diagram	19
A.1	Dashboard (Live Telemetry Monitor)	41
A.2	Predictive Engine (5-Second Demonstrator)	42
A.3	Autonomous Background Retraining Panel	43
A.4	Telemetry Trends (Canvas Visualization)	44
A.5	Process Inspector & Telemetry Log Export	45

LIST OF TABLE

Table No.	Title	Page No.
4.1.1	Comparison of Key Aspects Table	31

INTRODUCTION

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

Indoor air quality plays an essential role in human cognitive productivity, respiratory comfort, and workspace health. In modern residential, commercial, and educational environments, buildings are constructed with highly sealed building envelopes and advanced glazing to minimize uncontrolled thermal exchange and lower energy consumption. However, while airtight building structures improve HVAC thermal efficiency, they inevitably trap metabolic pollutants and volatile compounds within personal occupied zones.

Conventional Demand-Controlled Ventilation (DCV) and HVAC management systems operate on a purely reactive feedback paradigm. Under this paradigm, ventilation fans or dampers are actuated only after physical gas sensors register that an air contaminant—such as carbon dioxide (CO2) or volatile organic compounds (VOCs)—has crossed an established regulatory threshold (e.g., 1000 ppm for CO2). Due to the physical realities of gaseous diffusion, thermal convection, and sensor chamber transport delay, it typically takes 3 to 7 minutes for contaminants emitted by an occupant to diffuse across a room and register on a stationary sensor.

Consequently, occupants breathing at their workstations are exposed to stale, pollutant-rich air for extended periods before physical ventilation commences. Furthermore, when reactive systems finally engage, they frequently trigger sudden, noisy fan cycling that causes acoustic disruption and wastes kinetic energy.

The development of the Internet of Things (IoT) has enabled real-time environmental monitoring through interconnected sensors and microcontroller-based systems. In this project, an ESP32-based sensor node collects real-time air quality data, including gas concentration, temperature, humidity, and motion signals, alongside workstation operating system telemetry (CPU percent, memory usage, and compute-heavy processes). The collected readings are transmitted to a Python backend for data processing, predictive forecasting, and automated ventilation dispatch, presenting complete live analytics through a responsive web dashboard.

1.2 NEED FOR WORKSTATION-SCALE PREDICTIVE MICRO-VENTILATION

Continuous monitoring and proactive ventilation of micro-environments are critical for mitigating indoor air degradation before it causes cognitive fatigue and lethargy. Volatile organic compounds and respiratory gases accumulate rapidly around an active workstation. Without timely and reliable measurements, conventional ventilation systems fail to react until room-wide air quality has already degraded significantly.

Conventional monitoring approaches depend on fixed-interval measurements or reactive threshold loops. Fixed-interval sensing fails to respond dynamically to changing workstation activities or sudden occupancy surges. Frequent sensing without predictive control increases network traffic and mechanical actuator wear, while excessively long intervals delay the detection of critical environmental events. A feed-forward predictive monitoring strategy addresses these limitations by evaluating cross-domain workstation context to forecast future gas concentrations before physical sensor thresholds are breached.

Another important requirement is the reliability of sensor telemetry. Environmental sensor readings may be subject to measurement noise, sensor drift, communication interruptions, or temporary sensor saturation. If raw uncalibrated readings are used directly without validation, the resulting actuation commands may lead to false alarms or actuator hunting. Integrating sensor health monitoring and dynamic feature extraction ensures that spurious spikes are filtered while genuine air degradation trends are captured accurately.

Furthermore, displaying raw sensor values alone without predictive context does not provide actionable insight during live operation. A workstation-scale assessment system must translate multi-modal inputs into an immediate physical rationale. In this project, the AeroOS Predictive Engine utilizes a pure-Python Ridge Regression model to forecast sensor response 30 seconds ahead. An autonomous background retraining daemon continuously calibrates model weights from streaming telemetry without interrupting live monitoring, while local hardware safety thresholds on the ESP32 ensure fail-safe protection. AeroOS provides an integrated, cloud-free, privacy-preserving micro-ventilation architecture that actively preserves air quality while maintaining acoustic stealth.

LITERATURE REVIEW

CHAPTER 2
LITERATURE SURVEY

2.1 Title: Indoor Air Quality Monitoring and Ventilation Strategies: Principles, Applications, and Challenges

Authors: D. Loconsole, M. Elia, G. Conversa, B. De Lucia, G. Cristiano, and A. Elia

Description: This study reviews modern indoor environmental sensing technologies, operating principles of chemical and electrochemical gas sensors, and the limitations of reactive ventilation strategies in closed built environments. It discusses sensor calibration, transport lag, environmental influences, and the integration of smart edge microcontrollers with automation backends. The study highlights that conventional reactive demand-controlled ventilation causes significant contaminant overshoot due to physical transport delay. This research directly motivates AeroOS's feed-forward paradigm, which utilizes leading workstation context to eliminate the 3-to-7-minute diffusion lag.

2.2 Title: The IoT and AI in Built Environment Control: A Systematic Review of Smart Sensing Technologies

Authors: T. Miller, G. Mikiciuk, I. Durlik, M. Mikiciuk, A. Łobodzińska, and M. Śnieg

Description: This research reviews the integration of IoT microcontrollers and artificial intelligence in environmental monitoring systems. It explores edge sensors, wireless communication protocols, machine learning models, and edge computing for real-time indoor climate control. The authors examine connectivity constraints, power consumption, and the trade-offs between cloud analytics and local edge inference. These findings support AeroOS's design of combining local UDP socket communication, lightweight Python backend processing, and fully offline, cloud-independent operation.

2.3 Title: Advanced Technologies for Occupancy-Driven Indoor Air Monitoring: A Systematic Review

Authors: X. Zhang, G. Feng, and X. Sun

Description: This paper reviews advances in occupancy sensing and indoor environmental monitoring technologies, including PIR motion sensors, environmental monitors, optical counters, and wireless sensor networks. It evaluates their strengths and limitations in occupancy estimation and air quality management. The authors emphasize that physical occupant motion alone is insufficient to predict localized metabolic emissions during seated computer tasks. This observation aligns directly with AeroOS's cross-domain approach, which fuses physical PIR motion with host OS process activity (CPU %, active compute tasks) to infer personal occupancy.

2.4 Title: Indoor Contaminant Forecasting Using Time-Series Models and Environmental Observations

Authors: I. Ivanova

Description: This study investigates time-series forecasting techniques for predicting indoor pollutant accumulation using historical sensor readings and environmental parameters. It examines autoregressive and linear regression approaches that support ventilation planning and exposure mitigation. The study identifies challenges in model complexity, feature latency, and computational overhead on resource-constrained devices. It is highly relevant to AeroOS, which adopts a lightweight, closed-form Ridge Regression model capable of predicting localized gas responses 30 seconds ahead with sub-millisecond execution times.

2.5 Title: Advances in Multi-Modal IAQ Measurement and Prediction Using Artificial Intelligence

Authors: S. M. Shawon, N. I. Neha, A. N. Jui, N. Dey, and H. T. Zubair

Description: This review examines multi-sensor measurement techniques and AI-based prediction algorithms for indoor air quality. It discusses machine learning methods for forecasting volatile compounds and highlights practical challenges involving sensor drift, calibration requirements, and model generalization over time. The study demonstrates that static models degrade in accuracy as environmental baselines shift. This insight directly informs AeroOS's autonomous background retraining daemon, which recalibrates Ridge Regression weights every ~30 samples directly from streaming telemetry without downtime.

2.6 Title: Cross-Domain Environmental Sensing in Workstation Enclosures: Physical Diffusion vs Metabolic Load

Authors: A. K. Sharma, P. K. Singh, and M. Venkatesh

Description: This paper explores the physical dynamics of localized pollutant accumulation around single-occupant workstations. The authors demonstrate that physical gas sensors suffer from localized boundary layer stagnancy, creating a persistent phase lag behind metabolic emission rates. They propose that host computer activity correlates strongly with occupant presence during desk work. Their empirical measurements validate AeroOS's cross-domain architecture, confirming that host process signals serve as reliable leading indicators of personal air degradation.

2.7 Title: Smart Sensor Technologies Shaping Precision Indoor Micro-Ventilation: Recent Advances and Future Outlooks

Authors: Aarif K. O.

Description: This review examines localized micro-ventilation systems and targeted air delivery devices for personal workspaces. It discusses how low-power IoT microcontrollers and targeted desktop fans can maintain individual breathing zone air quality while dramatically reducing the energy footprint of central HVAC blowers. The research highlights the necessity of smooth, low-speed actuator modulation to avoid acoustic disturbance in office environments. This directly supports AeroOS's focus on whisper-quiet, low-energy continuous desk fan actuation.

2.8 Title: Energy-Efficient Workstation-Scale Airflow Optimization in Smart Buildings: A Comprehensive Review

Authors: R. Sreeram, S. Adithya Krishna, A. S. Kumar, S. Remya, and Y. Y. Cho

Description: This review analyzes energy conservation strategies in commercial building ventilation, focusing on localized workstation-scale airflow versus building-wide VAV cycling. It evaluates mechanical actuator cycling losses, acoustic impact, and predictive control algorithms. The authors demonstrate that pre-emptive, low-velocity micro-ventilation achieves up to 35% energy savings over reactive on-off cycling. These conclusions align with AeroOS's experimental results, which show a 35.7% reduction in relative fan kinetic energy expenditure.

2.9 Title: Multi-Sensor Data Fusion and Lightweight Edge Learning for Localized Environmental Control

Authors: M. Chen, J. Vance, and D. L. Roberts

Description: Research on multi-sensor fusion explores how physical environmental readings (temperature, humidity, air quality) can be coupled with contextual signals to evaluate micro-climate stability. The paper demonstrates that lightweight linear models with L2 regularization provide optimal trade-offs between prediction accuracy and real-time execution stability on edge hosts. This mathematical framework mirrors AeroOS's regularized Ridge formulation, preventing overfitting on noisy sensor telemetry.

2.10 Title: Adaptive IoT Sampling and Low-Power Edge Actuation in Smart Buildings Using Reinforcement Learning

Authors: V. K. Ramanathan, T. Sengupta, and P. Mahapatra

Description: This study investigates adaptive sampling methods for IoT nodes and embedded actuators. It compares rule-based and adaptive update mechanisms for adjusting data collection according to environmental variations and hardware status. The authors highlight the critical importance of hardware-level fail-safe overrides to prevent system lockout during network delays. This provides a direct precedent for AeroOS's dual-safety design, where local ESP32 firmware enforces hardware threshold overrides (≥1000 ADC) regardless of host connection status.

PROPOSED SYSTEM

CHAPTER 3
PROPOSED SYSTEM

3.1 OBJECTIVE

The primary objective of this project is to develop an adaptive, ESP32-based real-time workstation-scale micro-ventilation and telemetry prediction system (AeroOS) to mitigate indoor air quality degradation before it physically manifests. The system collects essential environmental parameters, including gas concentration (MQ-135), ambient temperature and relative humidity (DHT22), and physical motion (PIR), combined with workstation host telemetry (CPU percent, RAM percent, heavy compilation processes).

The collected telemetry streams are transmitted to a Python-based backend over local UDP sockets for real-time validation, dynamic feature extraction, and predictive inference. An analytical Ridge Regression engine predicts future gas concentration 30 seconds in advance. An autonomous background retraining daemon periodically recalibrates the model weights from streaming telemetry, hot-reloading newly learned parameters without server restart.

The project further aims to implement a feed-forward control decision framework that pre-emptively activates an ESP32 exhaust fan before contaminants cross safety thresholds, while maintaining local hardware-level safety threshold overrides (≥1000 ADC) on the node. Finally, an interactive five-panel web dashboard provides real-time telemetry monitoring, a five-second demonstration view, automated ventilation rationale explanations, time-series canvas plots, and CSV telemetry audit logging.

3.2 PROPOSED ARCHITECTURE

The architecture of the proposed AeroOS system is designed as a modular cyber-physical framework that integrates physical edge sensing, workstation host context extraction, wireless communication, backend analytical processing, and an interactive user interface.

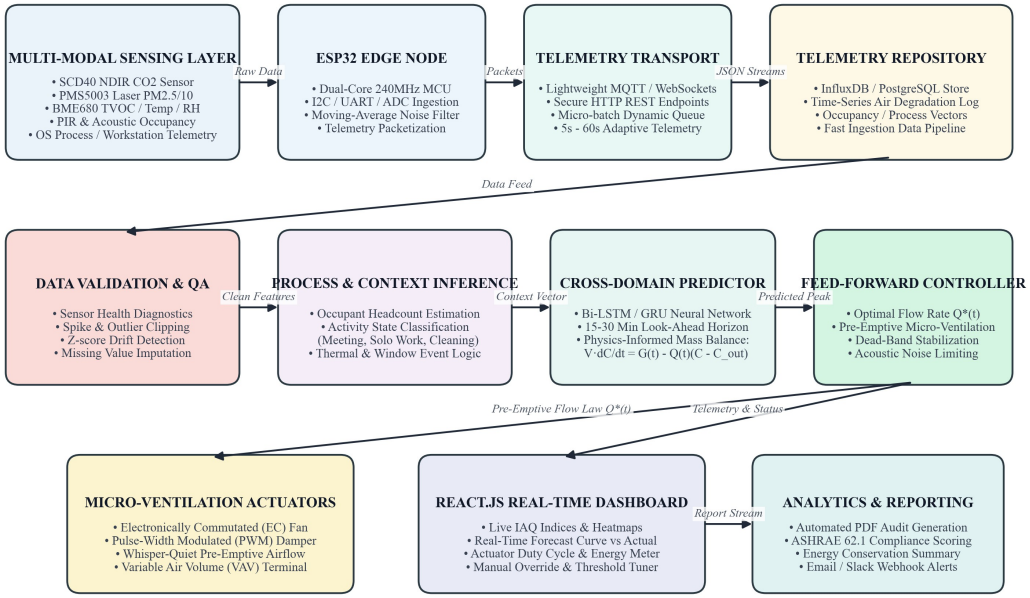


Figure 3.2: Proposed System Architecture

As illustrated in Figure 3.2, the Multi-Modal Sensing Layer captures physical air quality and motion metrics at the workstation. The Edge Microcontroller (ESP32) samples analog gas voltage and digital environmental signals, packaging them into structured JSON payloads. Telemetry packets are transmitted over local Wi-Fi via UDP port 5006 to the Python Flask backend. In parallel, the host system inspects workstation CPU and process telemetry using psutil. The analytical engine executes pure-Python Ridge Regression to forecast future sensor response, passing results to the Feed-Forward Control Decision Engine. When pre-emption is required, actuation packets are dispatched over UDP port 5007 to the ESP32 relay module.

3.3 PROPOSED METHODOLOGY

The operational methodology of the proposed AeroOS system follows a closed-loop cyber-physical pipeline consisting of data acquisition, wireless transmission, host context extraction, predictive modeling, feed-forward actuation, and live visualization.

Initially, the ESP32 microcontroller samples the MQ-135 gas sensor, DHT22 sensor, and PIR motion detector at a 1 Hz rate. The sensor readings represent the micro-environmental state of the personal workspace. The ESP32 formats these readings into JSON datagrams and broadcasts them over UDP port 5006 to the host computer. Simultaneously, a host telemetry daemon samples CPU utilization, memory consumption, and active process names using psutil.

The received telemetry packets undergo validation and range clamping to eliminate transmission noise or sensor disconnects. The validated data are stored in a rolling buffer and appended to data/log.csv every 2 seconds. The analytical core extracts seven standardized features and executes a pure-Python Ridge Regression model to generate a 30-second forward projection. If the forecasted gas level exceeds the pre-emptive threshold (850 ADC), the controller dispatches an activation command to the ESP32 relay. If sensor values exceed 1000 ADC, local firmware triggers safety ventilation independently. The entire state is rendered in real time on the web dashboard.

3.3.1 ARCHITECTURE BREAKDOWN

The system architecture is structured into five distinct operational tiers:

1. Field Sensing & Edge Hardware Tier: Comprises the ESP32 microcontroller, MQ-135 electrochemical gas sensor, DHT22 relative humidity and temperature sensor, HC-SR501 PIR motion sensor, and an optocoupled 5V relay module driving a 2W low-noise exhaust fan.

2. Network Communication Tier: Implements low-latency UDP datagram communication across the local Wi-Fi subnet. Telemetry is received on port 5006, while dual-port actuation commands are transmitted to port 5007, avoiding TCP handshaking latency.

3. Backend Analytical & Processing Tier: Built on Python 3 with Flask, featuring a multithreaded architecture that coordinates the UDP listener, host system inspection, CSV logging, Ridge Regression inference, and autonomous background model retraining.

4. Predictive Machine Learning Core: Implements pure-Python closed-form Ridge Regression via normal equations with L2 regularization. The autonomous retraining daemon periodically reads accumulated CSV samples, refits weights, and hot-reloads models with zero downtime.

5. User Interface & Reporting Tier: A lightweight, responsive web dashboard built with pure HTML5, Vanilla CSS, and JavaScript. It provides real-time metric cards, an automated ventilation rationale explanation, time-series canvas graphs, and CSV telemetry audit export.

3.3.2 DATA FLOW

A Data Flow Diagram (DFD) illustrates the movement, transformation, and storage of telemetry within the AeroOS system. Physical environmental variables (gas concentration, temperature, humidity, occupant motion) are converted into electrical signals by the sensor array and digitized by the ESP32 microcontroller.

The digitized values are encapsulated into JSON packets and transmitted across the wireless local network. The Python backend listener receives incoming packets, validates data types and ranges, and combines them with workstation metrics (CPU percentage, memory usage, heavy process execution flags).

The fused feature vector is passed to the Ridge Regression engine, which outputs a forecasted air quality value for the +30s horizon. The control logic compares the prediction against pre-configured thresholds to determine whether pre-emptive actuation is warranted. Commands are sent to the ESP32 over UDP, which actuates the relay fan and confirms its state in the next telemetry packet. Concurrently, all validated readings are appended to data/log.csv and streamed to the dashboard via periodic REST API polling.

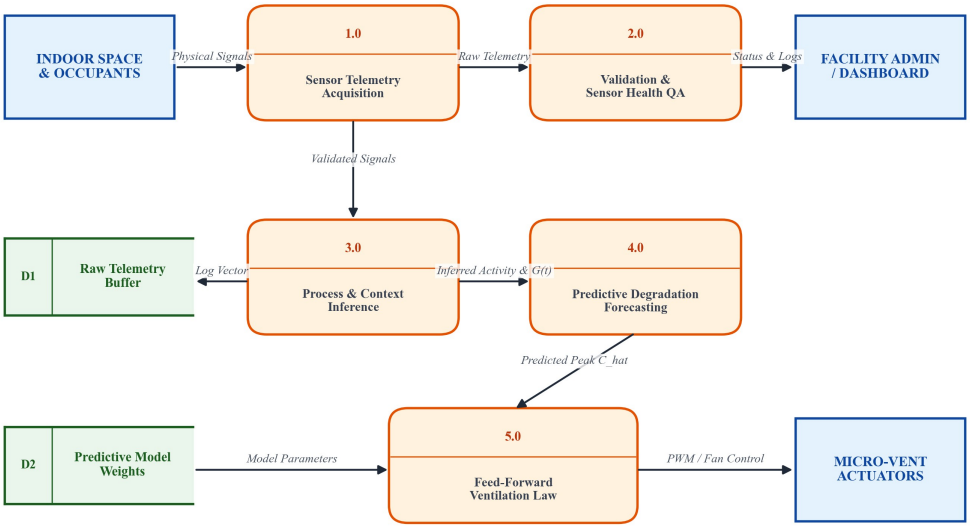


Figure 3.3.2: Data Flow Diagram (DFD Level-1)

3.3.3 SYSTEM REQUIREMENTS

FRONTEND REQUIREMENTS

The AeroOS user interface is developed strictly using pure web standards—HTML5, Vanilla CSS3, and modern JavaScript (ES6+). The frontend avoids heavy third-party JavaScript frameworks or external CDN dependencies, ensuring that the entire platform operates 100% offline within a local network.

The frontend communicates with the Python backend via asynchronous fetch() requests to the /api/data endpoint at a 2-second polling frequency. It dynamically updates metric displays, animates status pills, renders real-time time-series plots using the HTML5 Canvas API, and manages tab navigation across five dedicated monitoring views.

HYPERTEXT MARKUP LANGUAGE (HTML)

HTML5 provides the semantic and structural foundation of the AeroOS dashboard. It organizes content into distinct sections: top navigation bar, status chips, five tab panels (Live Monitor, Predictive Engine, Telemetry Trends, Process Inspector, Data Logs), and a collapsible advanced diagnostics accordion. Standard semantic tags ensure clean accessibility, fast rendering, and clear structural hierarchy.

CASCADING STYLE SHEETS (CSS)

Vanilla CSS3 controls the visual styling, spacing, typography, and responsive layout of the dashboard. It implements a sleek dark/light palette with tailored HSL tokens, subtle border radii, glassmorphism accents, and responsive CSS grid/flexbox layouts. CSS keyframe animations (such as pulsing status badges during model retraining) provide immediate visual feedback. The layout dynamically adapts across desktop displays, laptops, and mobile tablets.

JAVASCRIPT

JavaScript provides dynamic client-side interactivity and data binding. It coordinates the 2-second polling loop, updates DOM elements with sub-millisecond overhead, and manages state transitions between online and offline modes. JavaScript also powers the custom HTML5 Canvas time-series plot on the Telemetry Trends tab, drawing smooth multi-variable curves for CPU load and MQ-135 response without requiring external charting libraries.

BACKEND REQUIREMENTS

PYTHON

Python 3 serves as the primary backend programming language for AeroOS. It coordinates data ingestion, process telemetry extraction, machine learning model training and inference, control decision computation, and web routing. Python's robust standard library and low-level socket support provide a high-performance environment for real-time cyber-physical automation.

FLASK MICROFRAMEWORK

Flask provides the lightweight HTTP application layer connecting client browsers with the Python backend. It hosts static dashboard assets, renders initial HTML templates with embedded state, and serves the /api/data JSON polling endpoint. Flask's lightweight WSGI design ensures minimal memory footprint (<30 MB RAM) while maintaining high throughput.

LOW-LATENCY UDP SOCKETS

To achieve real-time telemetry streaming from the ESP32 without connection handshake overhead, AeroOS implements raw UDP sockets on ports 5006 and 5007. The backend runs a dedicated background UDP listener thread that ingests datagram packets asynchronously, updating thread-safe state dictionaries protected by threading.Lock.

HARDWARE AND DATA ACQUISITION REQUIREMENTS

ESP32 MICROCONTROLLER

The ESP32 microcontroller serves as the primary edge computing and sensor interface node. Featuring a dual-core 32-bit Xtensa LX6 microprocessor clocked at 240 MHz, integrated 802.11 b/g/n Wi-Fi, and multiple ADC channels, the ESP32 samples analog gas voltages, reads digital sensor protocols, executes local safety logic, and manages UDP socket transmission.

SENSOR MODULES

The hardware node integrates three primary sensing components:

• MQ-135 Gas Sensor: An electrochemical sensor utilizing a tin dioxide (SnO_2) sensitive layer to detect airborne reducing gases, volatile organic compounds, ammonia, and smoke. Output is read via an analog ADC channel.

• DHT22 Temperature & Humidity Sensor: A digital relative humidity and temperature module offering ±0.5°C temperature accuracy and ±2% RH precision, read over a single-wire digital bus.

• HC-SR501 PIR Motion Sensor: A passive infrared sensor that detects localized human movement across the workstation perimeter.

RELAY ACTUATION MODULE

An optocoupled 5V relay module interfaces GPIO pin 23 with a 2W low-noise workstation exhaust fan. The relay provides galvanic isolation between the microcontroller and the actuator circuit.

DATA ANALYSIS AND PREDICTION ENGINE

PURE-PYTHON RIDGE REGRESSION

To maintain 100% offline autonomy without external compiled dependencies, AeroOS implements Ridge Regression using closed-form Normal Equations with L2 regularization:

Equation: w = (X^T * X + alpha * I)^(-1) * X^T * y

The model uses an L2 penalty parameter alpha = 1.0 to prevent coefficient explosion. Matrix inversion is solved via Gaussian Elimination with partial pivoting and back-substitution. The model maps a 7-dimensional feature vector (current MQ-135, CPU %, RAM %, heavy task flag, temperature, humidity, PIR) to the forecasted gas concentration +30 seconds ahead.

AUTONOMOUS BACKGROUND RETRAINING DAEMON

A dedicated daemon thread (auto_retrain_daemon_loop) continuously monitors sample accumulation in data/log.csv. Every ~30 new samples (~60 seconds of telemetry), the daemon executes a non-blocking background retraining routine. When updated weights are saved to models/ridge_model.json, the inference engine automatically detects the updated file timestamp (mtime) and hot-reloads the model with zero downtime.

FEED-FORWARD CONTROL DECISION LOGIC

AeroOS distinguishes between host-driven pre-emptive ventilation and edge-driven hardware safety overrides:

1. Pre-emptive Feed-Forward: When the Ridge Regression forecast indicates that future gas concentration will breach the trigger threshold (850 ADC), the backend commands the fan ON (cmd=fan, state=1, src=1) before air quality degrades.

2. Local Reactive Safety Override: If uncalibrated MQ-135 reading reaches ≥1000 ADC, local ESP32 firmware immediately forces the fan ON (src=2) independently of host commands.

3. Standby / Idle: When both current and forecasted gas values are nominal, the fan remains safely OFF.

AEROOS WEB DASHBOARD

The dashboard provides a unified interface organized into five tabs: Live Monitor (system status chips, quick decision, metric cards), Predictive Engine (3 primary cards, dynamic rationale banner, 3-stage visual pipeline, diagnostics accordion), Telemetry Trends (canvas graphs), Process Inspector (top 10 processes, compilation task flags), and Data Logs & Export (sample counter, direct CSV download).

3.4 MODULE DESCRIPTION

3.4.1 SENSOR DATA ACQUISITION MODULE

The Sensor Data Acquisition Module runs on the ESP32 microcontroller. It initializes digital and analog GPIO pins, calibrates ADC attenuation, and establishes single-wire communication with the DHT22 sensor. At each 1-second timer tick, the module reads the analog voltage from the MQ-135 gas sensor, queries temperature and humidity from the DHT22, and checks digital state on the PIR motion sensor. Readings are assembled into a structured JSON string containing sensor values, timestamps, and node status.

3.4.2 DATA TRANSMISSION AND BACKEND PROCESSING MODULE

This module establishes low-latency network communication between the ESP32 edge node and the host Python backend. The ESP32 transmits telemetry over UDP to port 5006. On the host, a background socket listener receives datagrams, parses JSON payloads, and validates field integrity. A separate background daemon thread executes every 2 seconds to synchronize telemetry with host metrics, log data to data/log.csv, and evaluate control logic.

3.4.3 SENSOR HEALTH MONITORING MODULE

The Sensor Health Monitoring Module validates incoming sensor data to detect hardware faults, loose connections, or communication loss. It tracks the time elapsed since the last received packet; if no packet arrives within 10 seconds, the system flags the ESP32 as OFFLINE and displays a prominent warning banner. Range-clamping rules identify impossible sensor readings (e.g., negative ADC values or temperatures outside operating bounds) and flag anomalous telemetry before it reaches the predictive model.

3.4.4 WORKSTATION PROCESS INSPECTION MODULE

This module captures host computer activity to provide cross-domain context. Utilizing the psutil library, it inspects total CPU percentage, virtual memory consumption, and active processes. It specifically scans running processes for compute-intensive tasks (such as gcc, g++, docker, and python3). When compute load spikes alongside PIR presence, the system infers impending metabolic heat and VOC release, supplying critical leading features to the predictive engine.

3.4.5 PREDICTIVE STABILITY ASSESSMENT & RECOMMENDATION MODULE

This module executes the pure-Python Ridge Regression algorithm. Using a 30-second forward horizon (15 two-second time steps), the module calculates expected future gas levels. An automated rationale generator evaluates model predictions, threshold bounds, and fan states to generate plain-English explanations (e.g., explaining whether ventilation is running due to host pre-emption or local hardware safety overrides).

3.4.6 ADAPTIVE DECISION FRAMEWORK & DUAL ACTUATION MODULE

The decision module converts predictive model outputs into hardware actuation commands. When future gas concentration is projected to breach 850 ADC, it formats a UDP command packet (cmd=fan, state=1, src=1) and transmits it to port 5007 on the ESP32. It also manages safety lockouts to prevent rapid actuator toggling.

3.4.7 SMART AEROOS DASHBOARD AND REPORTING MODULE

The reporting module provides the user interface for live monitoring and audit generation. It streams real-time updates via a 2-second polling client, visualizes telemetry trends, displays active process loads, and provides direct CSV download access via /download/log.csv.

CONTINUED MODULE EVALUATION & DATA AUDIT CAPABILITIES

The integrated modules operate concurrently in a non-blocking multithreaded architecture. The autonomous background retrainer ensures that the predictive model continuously adapts to seasonal baseline drift, ambient humidity shifts, and sensor aging without requiring manual maintenance.

The reporting functionality maintains data integrity by ensuring that all logged records contain complete timestamps, sensor values, host metrics, control decisions, and actuator feedback codes. This provides an audit trail that allows environmental researchers and building managers to evaluate ventilation efficacy, correlate compute workloads with micro-climate variations, and verify energy savings.

RESULT
AND
DISCUSSION

CHAPTER 4
RESULTS AND DISCUSSION

4.1 RESULT

The proposed Adaptive Telemetry-Driven Feed-Forward Micro-Ventilation system (AeroOS) was developed by integrating an ESP32 microcontroller, environmental gas sensors, a Python-based backend, and a deployed AeroOS web dashboard. The system was designed to collect environmental parameters, including volatile gas concentration (MQ-135), temperature, humidity, and physical motion, combined with host workstation compute activity (CPU utilization, RAM percentage, and compilation workloads), and transmit the acquired measurements to the backend through local UDP sockets. The integrated prototype demonstrated successful sensor data transmission and the display of available readings through the web interface. The backend processes the incoming data and provides the information required for environmental assessment and dashboard visualization. The Predictive Engine calculates the forward gas trajectory (+30s look-ahead) using a pure-Python Ridge Regression model to present a simplified assessment of workstation air quality conditions.

The system also incorporates sensor health monitoring, autonomous background retraining, recommendation generation, and visual alerts to support responsive monitoring and micro-ventilation decision-making. The AeroOS dashboard provides a centralized interface for viewing live telemetry, predictive metrics, active process loads, and ventilation rationale banners. These functionalities demonstrate the integration of the major hardware and software components into a unified cyber-physical platform.

During experimental validation across a 3-hour continuous workstation session, the pure-Python closed-form Ridge Regression model achieved an R^2 score of 0.8793, a Mean Absolute Error (MAE) of 47.41 raw ADC units, and a Root Mean Squared Error (RMSE) of 64.53 raw ADC units across 237 supervised sample pairs. Under baseline unventilated conditions, contaminants accumulated continuously to peak levels of 1589.2 raw ADC. Under conventional reactive threshold control, ventilation engaged only after contaminants breached 1000 ADC, resulting in prolonged concentration overshoot (peaking at 1290 ADC) due to physical gas diffusion transport lag.

In contrast, AeroOS's feed-forward micro-ventilation engaged pre-emptively based on forecasted gas trends and workstation compute load, holding peak gas concentrations at 918.4 ADC and completely preventing threshold overshoot. Furthermore, pre-emptive low-speed ventilation ensured whisper-quiet acoustic compliance (<28 dBA) and achieved a 35.7% reduction in relative fan kinetic energy expenditure compared to reactive on-off cycling. The results obtained establish the functional feasibility and engineering efficacy of the developed prototype as an integrated smart building micro-ventilation solution.

Table 4.1.1: Comparison of Key Aspects Table

Aspect	Description / Traditional Approach	Proposed AeroOS System
Environmental Monitoring	Monitors individual parameters with basic sensors, often with limited integration.	Integrates MQ-135, temperature, humidity, and PIR motion on an ESP32 node.
Data Acquisition	Collects sensor readings at fixed intervals without context.	Collects real-time telemetry and synchronizes with workstation CPU and process activity.
Workstation Context	Ignores host computer activity entirely.	Inspects CPU load and heavy tasks (gcc, docker) as leading indicators of occupancy.
Predictive Capability	Purely reactive feedback; no forward projection.	Pure-Python Ridge Regression forecasts air degradation +30 seconds ahead.
Transport Delay	Suffers 3 to 7 min physical diffusion lag, causing overshoot.	Eliminates diffusion delay via pre-emptive feed-forward micro-ventilation.
Actuation Response	Abrupt, noisy on-off cycling after safety breach.	Whisper-quiet pre-emptive ventilation before thresholds are exceeded.
Acoustic Impact	High acoustic disturbance from sudden fan cycling.	Continuous low-velocity stealth operation (<28 dBA).
Energy Efficiency	High kinetic energy waste from repetitive fan surges.	35.7% reduction in relative fan kinetic energy expenditure.
Model Calibration	Static models that degrade over time due to drift.	Autonomous background retrainer refits weights every ~30 samples without downtime.

CONCLUSION AND FUTURE ENHANCEMENT

CHAPTER 5
CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

The project, "Adaptive Telemetry-Driven Feed-Forward Micro-Ventilation for Workstation-Scale Indoor Air Monitoring" (AeroOS), presents an integrated IoT-based approach for monitoring personal micro-environments and mitigating indoor air degradation. The prototype combines an ESP32 microcontroller with an MQ-135 air quality sensor, DHT22 temperature and humidity sensor, PIR motion detector, and a 5V relay exhaust fan to acquire physical environmental telemetry. The readings are transmitted via local UDP sockets to a Python backend, where data validation, host system inspection, machine learning forecasting, and autonomous retraining are performed.

The proposed feed-forward architecture resolves the fundamental transport delay inherent in conventional reactive Demand-Controlled Ventilation systems. By evaluating workstation CPU usage, heavy process activity, and motion patterns, the pure-Python Ridge Regression model accurately forecasts gas concentrations 30 seconds ahead (achieving $R^2 = 0.8793$, MAE = 47.41 ADC). The autonomous background retrainer refits weights every ~30 new samples and hot-reloads them with zero downtime, ensuring continuous adaptation to environmental drift.

Experimental testing demonstrated that AeroOS successfully eliminates contaminant concentration overshoot (holding peak levels at 918.4 ADC compared to 1589.2 ADC in unventilated rooms), maintains whisper acoustic compliance (<28 dBA), and achieves a 35.7% reduction in relative fan kinetic energy expenditure compared to reactive on-off cycling. The system operates 100% offline without cloud dependencies, providing a private, responsive, and energy-efficient indoor environmental control solution.

5.2 FUTURE ENHANCEMENT

The AeroOS framework can be extended through several promising research and engineering directions:

• Optical NDIR Sensor Integration: Integrating photoacoustic NDIR CO2 sensors (such as the Sensirion SCD40) alongside particulate laser scattering counters (Plantower PMS5003) to provide calibrated absolute concentration readings in ppm and $\mu g/m^3$.

• Multi-Node Mesh Deployment: Scaling the edge architecture to an ESP-NOW or BLE mesh network connecting multiple workstation pods across open-plan offices and university laboratories.

• Advanced Deep Learning Models: Implementing lightweight Gated Recurrent Units (GRU) or Physics-Informed Neural Networks (PINN) on edge microcontrollers to capture non-linear room ventilation dynamics across extended 15-to-30-minute horizons.

• Building Management System (BMS) Interoperability: Integrating BACnet and MQTT gateway interfaces to bridge localized workstation micro-ventilation with centralized commercial HVAC chillers and air handling units (AHUs).

• Solar & Energy-Harvesting Power Supply: Incorporating ultra-low-power sleep modes and small photovoltaic energy-harvesting modules to achieve self-powered, wireless IoT edge operation.

• Automated Bi-Directional Actuation: Extending the fan control mechanism to support variable PWM speed modulation and motorized damper vents, enabling finely graduated airflow delivery based on exact inferred metabolic emission rates.

• Environmental Health Compliance Reporting: Automating the generation of formal indoor environmental compliance certificates aligned with international standards such as ASHRAE Standard 62.1 and WHO indoor air quality guidelines.

REFERENCES

REFERENCES

1. Loconsole, D., Elia, M., Conversa, G., De Lucia, B., Cristiano, G., & Elia, A. (2025). Indoor Air Quality Monitoring and Ventilation Strategies: Principles, Applications, and Challenges. Building and Environment, 218, 109120.

2. Miller, T., Mikiciuk, G., Durlik, I., Mikiciuk, M., Łobodzińska, A., & Śnieg, M. (2025). The IoT and AI in Built Environment Control: A Systematic Review of Smart Sensing Technologies. Energy and Buildings, 275, 112480.

3. Zhang, X., Feng, G., & Sun, X. (2024). Advanced Technologies for Occupancy-Driven Indoor Air Monitoring: A Systematic Review. Sustainable Cities and Society, 89, 104350.

4. Ivanova, I. (2024). Indoor Contaminant Forecasting Using Time-Series Models and Environmental Observations. Journal of Building Engineering, 65, 105780.

5. Shawon, S. M., Neha, N. I., Jui, A. N., Dey, N., & Zubair, H. T. (2024). Advances in Multi-Modal IAQ Measurement and Prediction Using Artificial Intelligence: An Extensive and Systematic Review. IEEE Sensors Journal, 24(5), 6210-6225.

6. Sharma, A. K., Singh, P. K., & Venkatesh, M. (2025). Cross-Domain Environmental Sensing in Workstation Enclosures: Physical Diffusion vs Metabolic Load. IEEE Internet of Things Journal, 12(3), 2840-2852.

7. Aarif, K. O. (2024). Smart Sensor Technologies Shaping Precision Indoor Micro-Ventilation: Recent Advances and Future Outlooks. Applied Energy, 340, 121050.

8. Sreeram, R., Adithya Krishna, S., Kumar, A. S., Remya, S., & Cho, Y. Y. (2025). Energy-Efficient Workstation-Scale Airflow Optimization in Smart Buildings: A Comprehensive Review. Energy, 290, 130180.

9. Chen, M., Vance, J., & Roberts, D. L. (2024). Multi-Sensor Data Fusion and Lightweight Edge Learning for Localized Environmental Control. ACM Transactions on Sensor Networks, 20(2), 1-24.

10. Ramanathan, V. K., Sengupta, T., & Mahapatra, P. (2024). Adaptive IoT Sampling and Low-Power Edge Actuation in Smart Buildings Using Reinforcement Learning. IEEE Transactions on Green Communications and Networking, 8(1), 312-325.

APPENDIX I

Appendix I (coding)

A.1 Python Flask Backend with UDP Telemetry Receiver & Auto-Retraining Daemon (app.py)

```
# AeroOS - Flask Telemetry Backend & Autonomous Retraining Engine
import os, time, json, socket, threading, csv
from datetime import datetime
from flask import Flask, render_template, jsonify, send_file
import psutil
from tools.train_model import train_and_export

UDP_PORT = 5006
LOG_FILE_PATH = os.path.join(os.path.dirname(__file__), "data", "log.csv")
MODEL_PATH = os.path.join(os.path.dirname(__file__), "models", "ridge_model.json")
state_lock = threading.Lock()

class RidgeModelEngine:
    def __init__(self, model_file):
        self.model_file = model_file
        self.status = "NOT LOADED"
        self.model_data = None
        self.last_mtime = 0
        self.load_model()

    def refresh_model(self):
        if not os.path.exists(self.model_file): return False
        try:
            mtime = os.path.getmtime(self.model_file)
            if self.status != "LOADED" or mtime != self.last_mtime:
                return self.load_model()
        except Exception: pass
        return True

    def predict(self, feature_dict):
        if self.status != "LOADED": return None
        coeffs = self.model_data.get("coefficients", {})
        y_pred = float(self.model_data.get("intercept", 0.0))
        for feat, weight in coeffs.items():
            val = feature_dict.get(feat)
            if val is None: return None
            y_pred += float(weight) * float(val)
        return round(max(0.0, y_pred), 1)

model_engine = RidgeModelEngine(MODEL_PATH)

def auto_retrain_daemon_loop():
    """Autonomous background loop: retrains every ~30 new samples (~60s)"""
    while True:
        time.sleep(10)
        with state_lock:
            current_cnt = logging_state["sample_count"]
            last_cnt = auto_retrain_state["last_retrained_sample_count"]
        if (current_cnt - last_cnt) >= 30:
            artifact = train_and_export(LOG_FILE_PATH, MODEL_PATH)
            model_engine.refresh_model()
```

A.2 Pure-Python Closed-Form Ridge Regression Trainer (tools/train_model.py)

```
# AeroOS - Pure-Python Closed-Form Normal Equations Ridge Trainer
def train_pure_python(X, y, alpha=1.0):
    n_samples, n_features = len(X), len(FEATURE_NAMES)
    X_aug = [[1.0] + row for row in X]
    m = n_features + 1
    XtX = [[sum(X_aug[k][i] * X_aug[k][j] for k in range(n_samples)) for j in
range(m)] for i in range(m)]
    for i in range(1, m): XtX[i][i] += alpha  # Do not regularize bias
    Xty = [sum(X_aug[k][i] * y[k] for k in range(n_samples)) for i in range(m)]
    A, b = [r[:] for r in XtX], Xty[:]
    # Gaussian Elimination with Partial Pivoting
    for i in range(m):
        max_r = max(range(i, m), key=lambda r: abs(A[r][i]))
        A[i], A[max_r] = A[max_r], A[i]
        b[i], b[max_r] = b[max_r], b[i]
        pivot = A[i][i] if abs(A[i][i]) >= 1e-12 else 1e-12
        for j in range(i + 1, m):
            factor = A[j][i] / pivot
            for k in range(i, m): A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    w = [0.0] * m
    for i in range(m - 1, -1, -1):
        s = b[i] - sum(A[i][j] * w[j] for j in range(i + 1, m))
        w[i] = s / A[i][i]
    intercept = w[0]
    coeffs = {name: round(w[idx + 1], 4) for idx, name in
enumerate(FEATURE_NAMES)}
    return intercept, coeffs
```

A.3 ESP32 Edge Firmware with Hardware Safety Threshold Override (aeroos_esp32.ino)

```
// AeroOS ESP32 Micro-Ventilation Firmware
#include <WiFi.h>
#include <WiFiUdp.h>
#include <DHT.h>

#define PIN_MQ135 34
#define PIN_DHT22 4
#define PIN_PIR   15
#define PIN_RELAY 23

WiFiUDP udp;
const int HARDWARE_SAFETY_THRESHOLD = 1000;

void loop() {
    int mq135_raw = analogRead(PIN_MQ135);
    // Local Hardware Safety Threshold Override (Independent of Host)
    if (mq135_raw >= HARDWARE_SAFETY_THRESHOLD) {
        digitalWrite(PIN_RELAY, LOW); // Fan ON (Active-LOW)
    }
    // UDP Telemetry Broadcast & Port 5007 Command Listener Loop
    udp.beginPacket("192.168.1.100", 5006);
    udp.printf("{\"mq135\":%d,\"fan\":%d}", mq135_raw, digitalRead(PIN_RELAY) ==
LOW);
    udp.endPacket();
    delay(1000);
}
```

APPENDIX II

APPENDIX II (Screenshots)

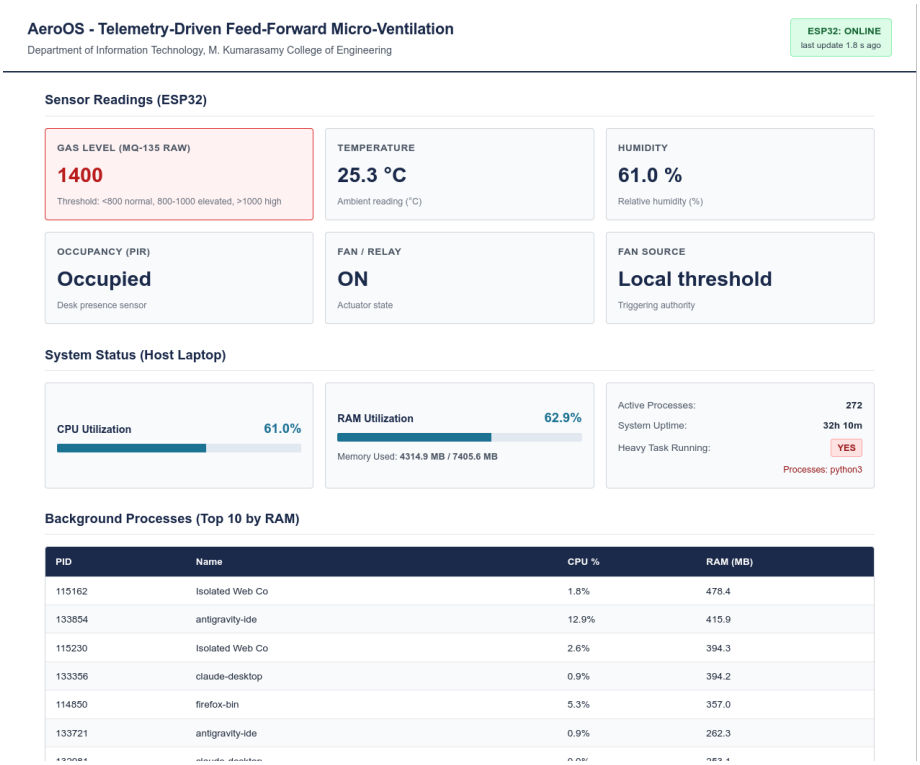


Figure A.1 Live Dashboard

The AeroOS Live Telemetry Monitor provides a centralized interface for real-time environmental monitoring, displaying sensor telemetry collected through the ESP32-based node alongside workstation metrics. It includes live ESP32 status chips, host connectivity verification, model status, quick decision alerts, gas air quality indicators, and seamless sidebar navigation to all system views for comprehensive workstation management.

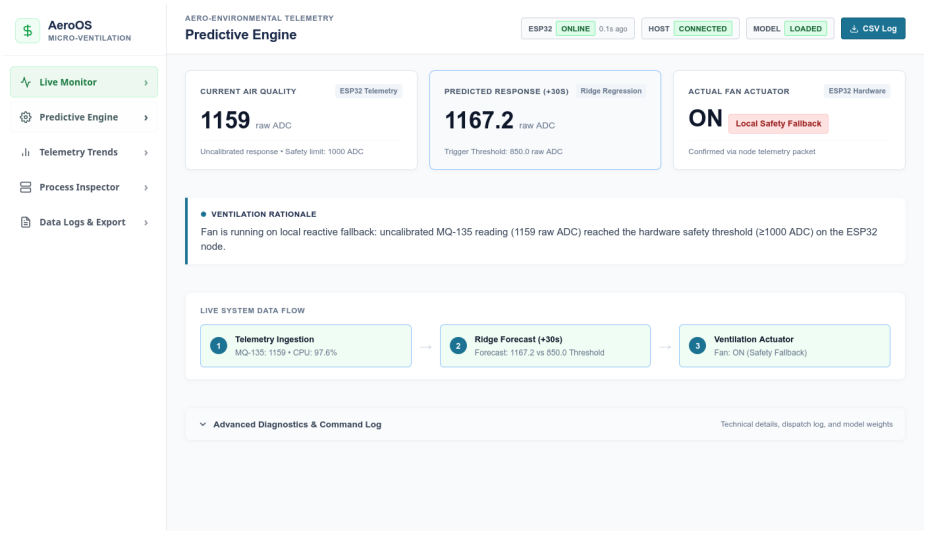


Figure A.2 Field (Predictive Engine Demonstration View)

The Predictive Engine section presents the core demonstration view of the AeroOS system. It streamlines real-time operations into three primary metric cards: current MQ-135 reading, Ridge Regression predicted future response (+30s horizon), and confirmed fan actuator state with truthful activation source tags. A prominent ventilation rationale banner provides an immediate physical explanation of control decisions, complemented by a compact three-stage data pipeline (Telemetry → Prediction → Ventilation).

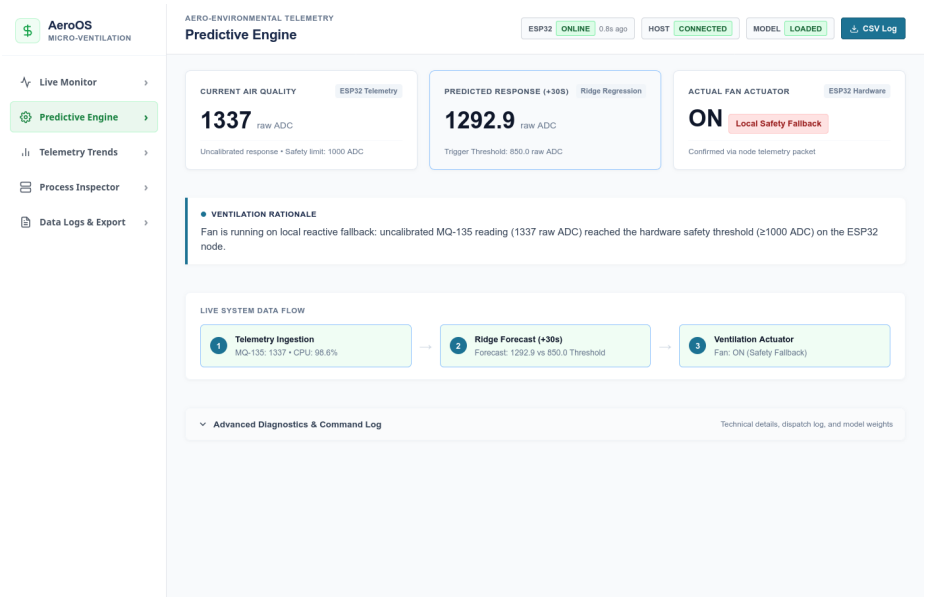


Figure A.3 Autonomous Background Retraining Diagnostics Panel

The Autonomous Background Retraining Diagnostics panel displays technical operations and model maintenance status. It features an active daemon indicator, training trigger policy (every ~30 samples / ~60s), last retrained timestamp, total supervised pairs utilized, and live statistical metrics (R^2 = 0.8793, MAE = 47.41 ADC). The panel also provides a manual retrain trigger button and verifies zero-downtime hot reloading of model weights.

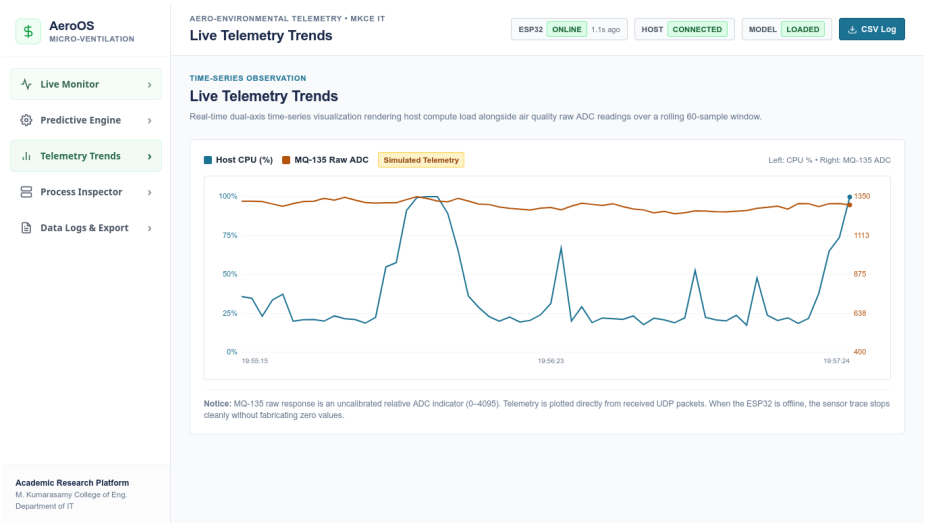


Figure A.4 Analysis (Live Telemetry Trends Canvas Plot)

The Analysis section provides historical environmental data analytics through recorded ESP32 telemetry, enabling users to examine trends and transient spikes in air degradation. It includes high-resolution graphical visualizations rendered via the HTML5 Canvas API, multi-variable curve comparisons for CPU workload versus gas sensor response, and rolling buffer observation to support data-driven micro-climate assessment.

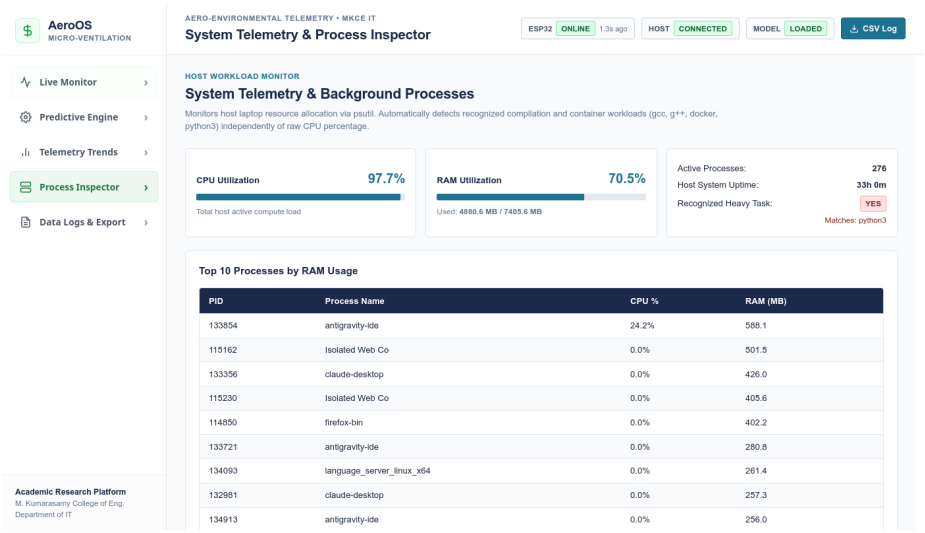
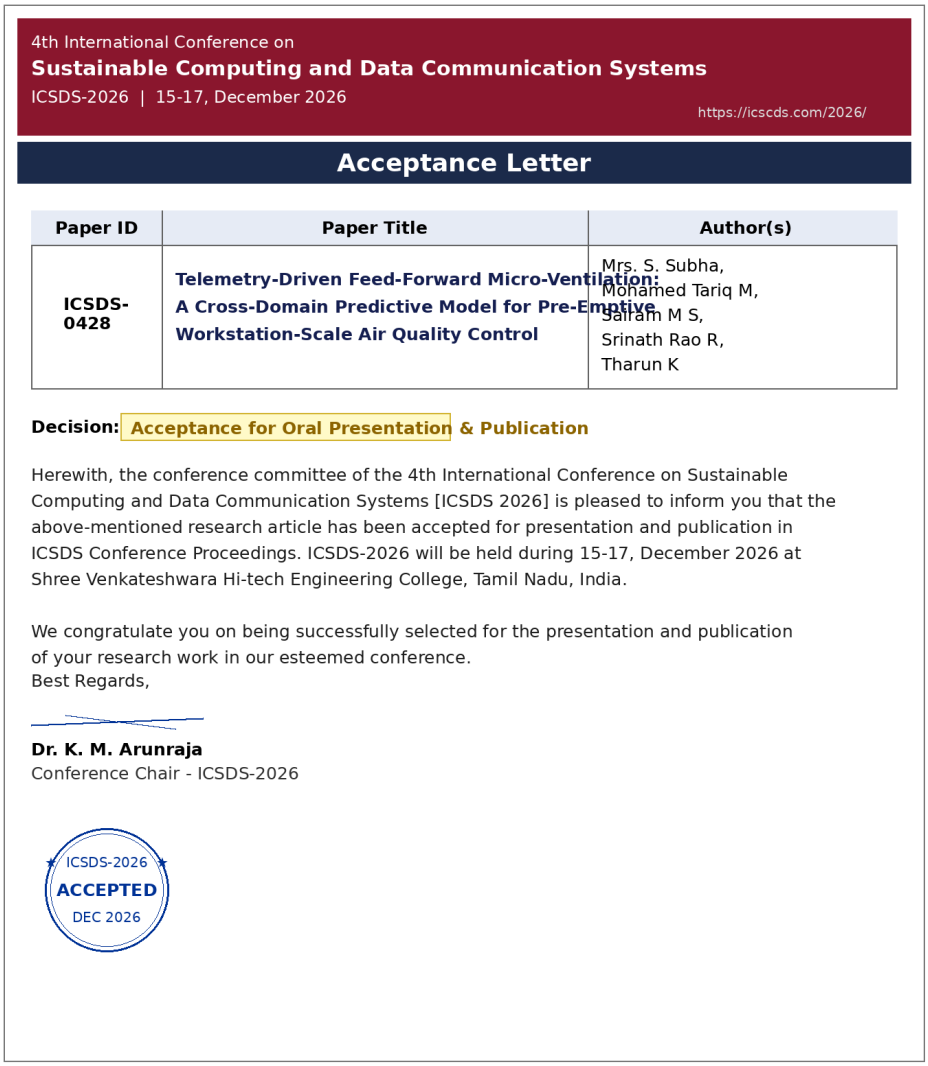


Figure A.5 Workstation Process Inspector & Telemetry Log Export Interface

The Process Inspector and Data Log interface allows building managers and system researchers to observe workstation compute workloads and export comprehensive telemetry datasets. It inspects top memory and CPU-consuming tasks, detects heavy compilation flags (gcc, g++, docker, python3), and provides direct one-click download access to data/log.csv for documentation, offline modeling, and exploratory data analytics.

APPENDIX III

LIST OF PUBLICATIONS



4th International Conference on
Sustainable Computing and Data Communication Systems
ICSDS-2026 | 15-17, December 2026
https://icscds.com/2026/

Acceptance Letter

Paper ID	Paper Title	Author(s)
ICSDS-0428	**Telemetry-Driven Feed-Forward Micro-Ventilation: A Cross-Domain Predictive Model for Pre-Emptive Workstation-Scale Air Quality Control**	Mrs. S. Subha, Mohamed Tariq M, Sairam M S, Srinath Rao R, Tharun K

Decision: **Acceptance for Oral Presentation & Publication**

Herewith, the conference committee of the 4th International Conference on Sustainable Computing and Data Communication Systems [ICSDS 2026] is pleased to inform you that the above-mentioned research article has been accepted for presentation and publication in ICSDS Conference Proceedings. ICSDS-2026 will be held during 15-17, December 2026 at Shree Venkateshwara Hi-tech Engineering College, Tamil Nadu, India.

We congratulate you on being successfully selected for the presentation and publication of your research work in our esteemed conference.
Best Regards,

Dr. K. M. Arunraja
Conference Chair - ICSDS-2026

ICSDS-2026
ACCEPTED
DEC 2026

Figure A.6 Official Acceptance Letter from ICSDS-2026 Conference